Improving Deployment Consistency of Cloud-Based Services: Manual Provisioning versus Terraform Automation

Meenakshi
24235415
Programme Code - Research in Computing CA1
National College of Ireland

1 Research Problem Background

Cloud computing has become the backbone of modern digital services, enabling organizations to deploy scalable applications across distributed environments. One of the major challenges in cloud service management is ensuring **deployment consistency**—the ability to reliably reproduce the same infrastructure configuration across multiple environments, such as development, testing, and production.

Traditionally, cloud infrastructure has been provisioned manually through provider consoles like AWS Management Console or Azure Portal. While manual provisioning may seem straightforward for small deployments, it often leads to configuration drift, human errors, and inconsistent infrastructure setups. These inconsistencies can cause service outages, security vulnerabilities, and increased operational costs Kasarla (2025).

In recent years, **Infrastructure as Code (IaC)** tools such as Terraform have emerged as a solution to these challenges. Terraform allows infrastructure to be defined in declarative configuration files, enabling automation, version control, repeatability, and faster recovery from failures Pohjola (2025). Despite Terraform's increasing adoption, many organizations still rely on manual provisioning due to lack of expertise or perceived complexity. Therefore, it is important to investigate whether Terraform automation significantly improves deployment consistency compared to manual provisioning approaches.

2 Research Question

To what extent does Terraform-based infrastructure automation improve deployment consistency, reliability, and error reduction in cloud-based service provisioning compared to manual provisioning methods?

3 Justification

This research question is significant because deployment inconsistency remains a major issue in cloud operations, especially for organizations adopting DevOps practices. Manual provisioning introduces risks such as:

- Unintentional configuration differences
- Increased deployment time
- Difficulty in reproducing environments
- Higher likelihood of human error

Terraform automation may address these problems by offering:

- Repeatable deployments
- Standardized infrastructure definitions
- Reduced configuration drift
- Improved operational efficiency

This research is worth investigating as it contributes to the growing field of cloud infrastructure automation and helps organizations make informed decisions about adopting IaC practices.

4 Specific Items to Be Addressed

This study will focus on the following key investigation areas:

1. **Manual Provisioning Challenges**

   Examining common issues such as inconsistency, drift, and error rates when infrastructure is created manually.

2. **Terraform Automation Benefits**

   Evaluating how Terraform improves repeatability, scalability, and infrastructure standardization.

3. **Experimental Deployment Comparison**

   Deploying identical cloud-based services using both methods and comparing outcomes based on performance metrics.

4. **Consistency Measurement Metrics**

   Defining measurable indicators such as configuration reproducibility, time efficiency, and failure frequency.

5. **Impact on DevOps Practices**

   Assessing Terraform's role in CI/CD pipelines and modern DevOps workflows.

6. **Recommendations for Cloud Organizations**

   Providing best-practice guidance for organizations transitioning from manual provisioning to IaC automation.

References

Kasarla, N. K. (2025), 'Implementing infrastructure as code (iac) with terraform for scalable cloud deployments', *Journal of Information Systems Engineering and Management* .

Pohjola, O. (2025), Mitigating configuration drift in infrastructure-as-code systems, Master's thesis, Aalto University.